

Distance-Preserving Digests: A Primitive for BFT Consensus^{*}

Ryan Mercier

School of Engineering, University of Connecticut, Storrs, CT, USA
ryan.mercier@uconn.edu

Abstract. Every BFT consensus protocol compares validator state with collision-resistant hashes, and collision resistance destroys distance: validators agreeing on 19 of 20 transactions produce unrelated hashes, indistinguishable from validators sharing nothing. This forces validators to synchronize state before voting, makes agreement quality unmeasurable until votes are counted, and requires hierarchical committees large enough for independent BFT. We introduce distance-preserving transaction digests: each transaction hashes to a point in an 8-dimensional space, a validator’s digest is the sum of the points for the transactions it holds, and the Euclidean distance between two digests is therefore proportional to the number of transactions on which the validators differ; weighted means summarize a group exactly, and set differences are identifiable via bloom filter diff. Three applications follow: a two-phase BFT protocol (Proxima) achieving single-round finality when validators agree, tree-structured consensus with groups of 10 validators (vs 128 in Ethereum), and cross-shard consistency verification at 128 bytes per shard pair. Safety is proved: fewer than $N/3$ Byzantine validators cannot cause conflicting finalization, independent of Phase 1 clustering or tree topology. At $N=100,000$, Proxima Tree uses $2.2\times$ fewer messages than HotStuff, and single-core finality is $\sim 0.9s$ vs $\sim 18s$.

Keywords: Byzantine fault tolerance · blockchain · consensus protocols · locality-sensitive hashing · sharding

1 Introduction

BFT consensus has been built on collision-resistant hashing since PBFT [5]. Collision resistance is the right property for preventing forgery but the wrong one for measuring agreement: differing hashes reveal one bit, not identical, whether the sets differ by one transaction or by half. In HotStuff [25], a leader-based BFT protocol used in production systems, a rotating leader drives each block through three voting rounds, collecting each round’s votes into a threshold-signature quorum certificate proving $2N/3$ support. Because the certificates are built from collision-resistant hashes, HotStuff runs all three rounds even when every validator agrees perfectly; the same one-bit comparison is why Ethereum 2.0 [12] needs

^{*} Code and reproduction artifacts: <https://github.com/RyanMercier/Proxima>

committees of 128 validators and why cross-shard transactions need multi-round coordination.

We propose distance-preserving digests for state comparison: hash each transaction with SHA-512, split the output into 8 segments, treat each as a coordinate, and sum across transactions, giving a commutative vector in 8D space whose distances are proportional to transaction disagreement. Locality-sensitive hashing is old [16]; its application to BFT consensus is, to our knowledge, new, and it removes all three constraints, which we demonstrate with Proxima, a working BFT blockchain.

Throughout, we analyze Proxima under a static adversary that corrupts validators before the protocol begins. Static corruption is the standard first-analysis setting for this protocol class, where PBFT, HotStuff, and Tendermint state their classical safety arguments, and it separates the primitive’s contribution from adaptive corruption, an orthogonal problem. Adaptive adversaries are out of scope; Sect. 9 discusses what defending against them requires.

2 Related Work

BFT Consensus. PBFT [5] established practical BFT at $O(N^2)$ messages; HotStuff [25] reached $O(N)$ via leader-based aggregation and threshold signatures, as does Tendermint [4]. All compare state with collision-resistant hashes and synchronize fully before voting; Proxima keeps $O(N)$ voting but adds distance-based pre-filtering and bloom-based synchronization. Its fast path follows the speculative lineage of Zyzzyva [17] and SBFT [15], where optimistically collected signed commitments finalize early when a quorum already agrees. DAG-based designs (Narwhal/Tusk [7], Bullshark [22]) decouple data availability from consensus; digests could measure vertex agreement across their partial views.

Hierarchical BFT. Kauri [19] applies tree aggregation to HotStuff, but its tree nodes still run per-group, hash-based BFT, requiring large groups; Ethereum 2.0 [12] committees of 128 exist for the same reason. Distance filtering removes the requirement, enabling groups of 10 and deeper trees (5 levels vs 2–3).

Locality-Sensitive Hashing and Sharding. LSH began with Indyk and Motwani [16] and spans SimHash [6], MinHash [3], and p -stable LSH [8]; ours is a simple LSH scheme for transaction sets, and neither the LSH nor the consensus literature has connected the two. On the sharding side, Ethereum moved to data-availability sharding (Danksharding [11]), NEAR’s Nightshade [21] uses receipts, and Shardeum shards by address range; digest verification is complementary, at constant cost per shard pair with resolution only for divergent transactions.

3 System Model

We consider N validators, at most $f < N/3$ of them Byzantine; honest validators follow the protocol. We assume partial synchrony [10]: after an unknown Global Stabilization Time (GST), messages between honest validators arrive within a known bound Δ . Each validator holds a BLS [2] key pair with public keys known

to all; signatures aggregate (N into one 96-byte aggregate), and we assume CDH hardness in BLS12-381 [23]. An aggregator (flat mode) or a tree of relays (tree mode) routes communication, rotating each round; a Byzantine aggregator can suppress messages (liveness) but cannot forge signatures (safety); the adversary is static, corrupting validators before the protocol begins. SHA-512 outputs are modeled as uniform, making each digest coordinate uniform in $[0, 1)$. Honest validators may receive transactions with delay relative to the proposer; p_{miss} , the per-validator probability of an incomplete view at proposal time, is 0.37 throughout, from internal measurements and conservative for typical mainnet conditions.

4 The Primitive: Distance-Preserving Digests

The digest must stay as cheap as an ordinary hash while turning set disagreement into measurable geometry. Each transaction tx is hashed with SHA-512; the 64-byte output is split into 8 segments of 8 bytes, each interpreted as an unsigned integer, reduced modulo 10000, and divided by 10000 to give a coordinate in $[0, 1)$, so that $v(tx) = [\text{seg}_0/10000, \dots, \text{seg}_7/10000]$. A validator’s digest for a transaction set T is the commutative sum $D(T) = \sum_{tx \in T} v(tx)$. Three properties follow that hashes lack. *Proportional distance*: sets differing by k transactions produce digests whose Euclidean distance grows proportionally with k , empirically ~ 1.6 at $k=1$, ~ 3.0 at $k=2$, ~ 14.5 at $k=10$. *Exact summarization*: the weighted mean of N digests represents the group exactly, so 100 validators summarize to one 76-byte value (64-byte weighted mean + 4-byte count + 8-byte variance). *Set difference identification*: when two digests differ, a bloom filter [1] (25 bytes at a 1% per-lookup false positive rate for 20 transactions) lets the aggregator identify exactly which transactions are missing and push them in one message; false negatives cannot occur, and false positives (Sect. 8) cost only liveness.

Threshold calibration and liveness bound. The distance threshold separating honest-with-partial-observation from Byzantine-with-fabricated-state is calibrated by Monte Carlo simulation: we generate 2000 samples of honest validators missing 1–2 transactions, take the 99th percentile of the distance distribution, and multiply by a safety margin of 1.2. Liveness requires the threshold to include at least $2N/3$ honest validators (safety needs only BLS signatures, Sect. 8), and the calibration makes each constant in the bound explicit. An honest validator misses at most $k_{\text{max}} = 2$ transactions (Sect. 3), exactly what the calibration samples. The 99th percentile of the sampled honest distances is approximately 4.1, so the 1.2 margin gives $\tau = 1.2 \cdot 4.1 \approx 4.9$; measured over the same sample, the fraction of honest draws exceeding 4.9 is at most 0.005, the exclusion probability conditioned on an incomplete view. Exclusion also requires the incomplete view itself (probability $p_{\text{miss}} = 0.37$), so the unconditional per-validator exclusion probability is at most $p = 0.37 \cdot 0.005 = 0.00185$. The number X of excluded honest validators is a sum of N independent indicators and liveness fails only if $X \geq N/3$, so Hoeffding’s inequality with $t = 1/3 - p \approx 0.33$ (so $2t^2 \approx 0.22$)

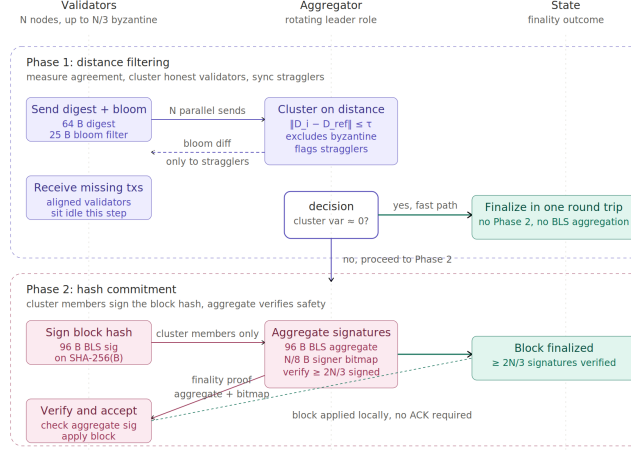


Fig. 1. Two-phase Proxima protocol with speculative fast path. Phase 1 messages carry digest, bloom, and, from each validator whose local state matches the proposal, a conditional BLS commitment (64 + 25 + 96 bytes); with $\geq 2N/3$ matching commitments collected in round one, the aggregator aggregates them into the finality certificate (96-byte aggregate plus $N/8$ -byte signer bitmap, the same object Phase 2 produces) and finalizes in one round trip.

gives $\Pr[X \geq N/3] \leq \exp(-2Nt^2) \leq \exp(-0.22N)$: about $3 \times 10^{-10} < 10^{-9}$ at $N=100$ and below 10^{-95} at $N=1000$.

5 Application 1: BFT Consensus Protocol

Proxima runs two phases: Phase 1 measures agreement geometrically and repairs stragglers, and Phase 2 converts agreement into finality through aggregated BLS commitments; when enough validators already hold the proposed block, the two collapse into a single round trip (Fig. 1). The signature phase can be accelerated but never skipped, because the distance filter cannot tell an honest validator missing one transaction from a Byzantine one that replaced it (both at distance ~ 1.6); only the Phase 2 signature on the correct block hash separates them.

Flat protocol. The proposer disseminates the block ahead of Phase 1; a validator misses transactions with probability p_{miss} (Sect. 3), so its local view may be incomplete. In Phase 1, each validator sends its digest (64 bytes) and bloom filter (25 bytes) to the aggregator (N messages), and a validator whose local state matches the proposal attaches to the same message a conditional BLS commitment σ_i on $(\text{height}, H(B_{\text{prop}}))$ (96 bytes); validators with incomplete views send digest and bloom only, so the Phase 1 payload is 64 + 25 + 96 bytes from matching validators and 64 + 25 bytes from the rest. The aggregator clusters validators by distance from the reference digest, pushes missing transactions to stragglers via bloom diff, and broadcasts cluster assignments.

If the aggregator collects at least $2N/3$ valid commitments on the same hash within round one, it aggregates them and multicasts the finality certificate immediately (fast path). The certificate is byte-identical to the object Phase 2 produces, a 96-byte aggregate signature plus an $N/8$ -byte signer bitmap, so single-round finality is covered by Theorem 1 with no modification. Otherwise the round falls back to full Phase 2 collection from all cluster members; commitments already received in Phase 1 may be reused there, a saving the evaluation conservatively does not model. The fast path applies to the flat protocol; carrying speculative commitments up the tree requires per-group signer bitmaps and is deferred. In Phase 2, cluster members send BLS-signed hash commitments (96 bytes each), and the aggregator multicasts the finality proof, the 96-byte aggregate plus $N/8$ -byte bitmap, to the cluster; finality requires $2N/3$ matching commitments.

Tree protocol. Leaf groups need no internal BFT and never fail: distance filtering operates at the individual level, so a group of 10 with 5 Byzantine excludes the 5 and reports the weighted mean of the rest. In Phase 1 (bottom-up), validators in leaves of size B (default 10) send digest and bloom to their leaf leader, which filters by distance, syncs stragglers by bloom diff, and sends a 76-byte summary upstream; internal nodes aggregate child summaries (weighted mean of means is exact), and the root checks the global mean against the reference. In Phase 2, filtered validators send BLS commits up, internal nodes aggregate them (BLS is associative), and the root broadcasts the finality proof back down.

Comparison with Ethereum committees. Ethereum 2.0 [12] uses random committees of 128 with per-committee 2/3 attestation; with 30% Byzantine globally, the chance of drawing more than $G/3$ Byzantine validators is $\approx 35\%$ at $G=10$ versus $\approx 21\%$ at $G=128$. Proxima has no per-group vote: in simulation with 1000 validators, 300 Byzantine, and groups of 10, per-group BFT loses 37 of 100 leaves, while distance filtering lets all 700 honest validators contribute; an all-Byzantine leaf’s fabricated summary still fails the root’s distance check.

Agreement quality and the fast path. When all validators have complete state, every Phase 1 message carries a matching commitment, so the aggregator collects the commitments in round one and issues the finality certificate immediately (Sect. 5). The fast-path probability is approximately $(1 - p_{\text{miss}})^N$; since this is the probability that every honest validator matches the proposal, while the certificate requires only $2N/3$ matching commitments, it is a lower bound on engagement. At $p_{\text{miss}} = 0.05$ and $N = 10$ honest validators (the leaf-group regime) the bound is $\approx 60\%$, rising to $\approx 90\%$ at $p_{\text{miss}} = 0.01$. Average distance from the reference over time also gives a continuous reputation signal: honest validators average 0.3–0.5, Byzantine validators 7.2.

6 Application 2: Cross-Shard Consistency

Cross-shard transactions bottleneck sharded execution: two-phase commit costs 4 cross-shard messages per transaction plus intra-shard BFT consensus on each

Table 1. Cross-Shard Overhead at 1000 txs, 95% propagation

Method	Messages	Bandwidth	X-shard
2PC	404,000	37,750 KB	4,000
Receipt (NEAR)	101,000	12,625 KB	1,000
Digest (95%)	5,052	987 KB	52

coordination message, and NEAR’s receipt-based Nightshade [21] achieves lower latency at the price of receipt volume that grows linearly with cross-shard transaction volume. Digests reduce the coordination to a constant-cost check: neighboring shards exchange digests of their overlap-zone transactions once per block, 128 bytes per shard pair. If the distance is zero, both shards processed the same transactions; if it is nonzero, the bloom diff identifies exactly which transactions diverged, and only those require resolution. Hash-based verification indicates only whether two states are identical, revealing nothing about the magnitude of divergence, so a mismatch forces a fallback to full state exchange or per-transaction coordination, while digest-based verification reports an approximate- k difference and identifies the divergent transactions for targeted resolution.

Table 1 compares the methods at 1000 cross-shard transactions per pair, 100 validators per shard, and 95% pre-deadline propagation: digest comparison reduces messages by 99% vs 2PC and 95% vs receipts, and its overhead scales with conflicts (5% of transactions) rather than volume, so the advantage persists at 100 shards (50,502 messages versus 4,040,000 for 2PC; per-method arithmetic in the extended version [18]).

7 Evaluation

The evaluation asks whether two phases and tree routing reduce message load, whether distributing BLS aggregation moves the latency bottleneck, and whether either costs Byzantine tolerance. Message counts come from a simulation counter incremented on every send, with identical byte constants across protocols.

Message complexity. Table 2 (top) reports messages per block with 30% Byzantine and 37% partial observation; all protocols grow linearly except PBFT, whose $O(N^2)$ growth reaches $\sim 20G$ messages at $N=100K$. Proxima Tree is $2.2\times$ fewer messages than HotStuff, from two phases instead of three, Byzantine exclusion before Phase 2 (700 commits instead of 1000), bloom sync replacing retransmission round-trips, and tree routing replacing broadcast with compact summaries.

BLS aggregation bottleneck. Latency is projected, not measured: we use published blst microbenchmarks (0.05ms per aggregate-add, 1.5ms per aggregate-verify) [23,20,9,14], plus a three-tier RTT model (1ms intra-rack, 80ms intra-region, 200ms cross-region), one tier-appropriate RTT per phase: 800ms for HotStuff’s four cross-region phases, 600ms for Flat’s three, and $2 \cdot (1 + 3 \cdot 80 + 200) = 882$ ms for the tree’s hop structure at $L=5$ (892ms in the table with accounting

Table 2. Messages per block (top) and projected finality latency at $N=100,000$ (bottom; the multi-core row is qualitative).

	Prox. Tree	Prox. Flat	HotStuff	PBFT
Msgs, $N=1K$	2,990	3,348	6,518	2.0M
Msgs, $N=10K$	30,042	33,600	65,180	200M
Msgs, $N=100K$	300,245	335,803	651,800	$\sim 20G$
BLS (1 core)	9.9 ms	3,960 ms	17,595 ms	
Network RTT (model)	892 ms	600 ms	800 ms	
Total finality (1 core)	902 ms	4,561 ms	18,395 ms	
BLS only (16 cores)	~ 10 ms	~ 220 ms	~ 940 ms	

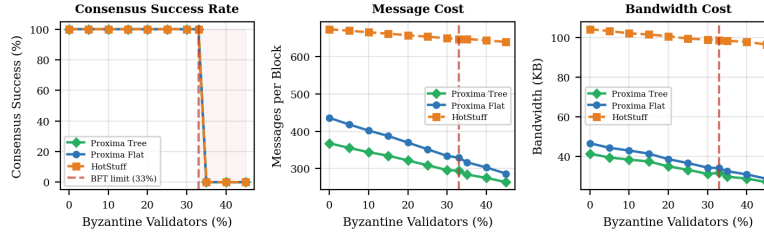


Fig. 2. Byzantine tolerance and cost scaling ($N=100$, 37% partial observation, 10 trials per point); Proxima’s costs decrease with Byzantine percentage because excluded validators skip Phase 2.

overhead). On a single core the flat aggregator spends 3.96s aggregating 70,000 BLS signatures and HotStuff roughly three times that, while the tree distributes aggregation across leaves so its critical-path BLS time is 9.9ms across four levels (Table 2, bottom); network RTT dominates tree finality while BLS dominates the others at large N .

Caveats. Multi-threaded baselines partially close the gap: BLS aggregation in flat protocols parallelizes well, so a 16-way split reduces flat aggregation to ~ 220 ms and each HotStuff round to ~ 313 ms, while the tree gains nothing from extra cores, each leaf already holding only ~ 7 signatures, and its 9.9ms requires leaf leaders on separate machines. The tree keeps its critical-path BLS advantage at any core count, but total multi-core finality also depends on whether non-BLS stages parallelize, unanswerable without a deployed testbed; the message-complexity and group-structure advantages are core-count-independent.

Byzantine tolerance. The system tolerates up to 33% Byzantine: in simulation, 100% consensus success from 0% to 33% and 0% at 35% and above (Fig. 2). As the Byzantine percentage increases, Proxima gets cheaper (fewer validators in Phase 2) while HotStuff’s cost is constant.

Implementation. Proxima is implemented in Python as a working blockchain with account-based state, an HTTP node API, an interactive demo, and benchmarks, reproducible from the repository [18]. Digest computation costs one

SHA-512 plus eight modular reductions per transaction (Sect. 4), and digests are vectorized sums traveling as 64 bytes on the wire. BLS signatures run through two interchangeable code paths: py-ecc [13] performs real BLS12-381 operations in the live demo, while large-scale benchmarks substitute hash-based mocks preserving the 96-byte signature and 48-byte public-key wire sizes; we cross-validated the paths by running the small- N demo under both and confirming identical per-block message totals. The implementation also synchronizes stragglers with a single bloom-diff push, aggregates speculative Phase 1 commitments directly into the fast-path certificate, and splits tree aggregation so each leaf leader combines only a branching factor of signatures.

8 Security Analysis

Safety rests entirely on signed commitments and holds no matter what Phase 1 does; every distance-related mechanism affects only liveness.

Theorem 1 (Safety). *If fewer than $N/3$ validators are Byzantine, Proxima never finalizes two different blocks at the same height.*

Proof. Finality requires $2N/3$ signed commitments on one block hash. Two conflicting certificates at the same height would intersect in at least $N/3$ validators, each having signed two different hashes at that height; honest validators sign at most one, so the intersection is entirely Byzantine, contradicting $f < N/3$.

Corollary 1. *Phase 1 cannot compromise safety. Incorrect inclusion of a Byzantine validator means it fails to produce a valid BLS signature. Incorrect exclusion of an honest validator reduces commit count (liveness) but no incorrect block is signed.*

The fast-path certificate is itself a set of at least $2N/3$ signed commitments on $H(B)$, collected in Phase 1 rather than Phase 2, so Theorem 1 applies to it verbatim: an aggregator cannot fabricate one and can at most withhold it, delaying the round (a liveness effect).

Tree routing likewise cannot compromise safety: BLS aggregation is associative, and a Byzantine tree node can suppress children’s signatures (liveness) but cannot forge signatures.

Liveness channels. Liveness holds when $f < N/3$, the threshold includes at least $2N/3$ honest validators (this fails with probability at most $\exp(-0.22N)$, Sect. 4), and messages are eventually delivered; a rotating aggregator with timeout ensures progress after GST.

Two further channels cost liveness, never safety. First, a randomly fabricated Byzantine digest lands within threshold τ of the reference with probability $\approx 4.06 \cdot (\tau/R)^8 \approx 9.1 \times 10^{-4}$ at $\tau = 4.9$, $R \approx 14$ (volume-ratio estimate; derivation in [18]); Monte Carlo over 10,000 trials shows 0.4%, and Phase 2 always catches the intruder (invalid BLS commitment). Second, a bloom false positive can leave a validator unsynchronized, so it exceeds τ , is excluded, and does not

sign in Phase 2; the chain is wholly liveness, since Theorem 1 depends only on collision-resistant hashing and BLS unforgeability, not on cluster membership. The 1% FP figure is per-lookup against $O(k)$ candidates (k typically 1–5), so the per-validator rate is $kp \approx 1\text{--}5\%$ (sizing analysis in [18]).

Adversarial transaction construction. A natural concern is whether a Byzantine validator can craft a transaction set $T' \neq T$ whose digest lands within τ of the honest reference $D(T)$. Distance-preserving digests are deliberately not collision-resistant; a reduction to SHA-512 collision resistance would prove too much. The construction is, at our parameters, computationally easy: each $v(tx)$ is determined by $\text{SHA-512}(tx)$, and the matching condition is roughly $8 \log_2(R/\tau) \approx 12$ bits, within laptop reach for Wagner-style k -list search [24]. *Phase 1 distance filtering therefore contributes no cryptographic security.* That is fine, because Phase 2 requires every cluster member to sign a BLS commitment to the SHA-256 block hash $H(B)$, and the Byzantine cannot forge a signature on $H(B)$ where B contains T ; Theorem 1 does not invoke any property of $D(\cdot)$. The construction buys the adversary no safety violation; the worst case is a failed Phase 2 round, indistinguishable from any other signature withholding.

9 Conclusion, Limitations, and Future Work

Collision-resistant hashing imposes three constraints on BFT consensus (mandatory state synchronization, unmeasurable agreement quality, large hierarchical committees); distance-preserving digests remove all three, with safety proved independently of the digest. At $N=100,000$, Proxima Tree uses $2.2\times$ fewer messages than HotStuff and reduces cross-shard overhead by 99% versus 2PC at 95% propagation; single-core finality is $\sim 0.9\text{s}$ vs $\sim 18\text{s}$. The primitive applies to any BFT protocol that compares state via hashes.

Four limitations bound these claims. **Projected latency:** all latency numbers use published blst benchmarks and standard RTT estimates; a deployed testbed across 3+ cloud regions would validate the model (multi-threading caveats in Sect. 7). **No comparison against hierarchical HotStuff:** comparing against Kauri would isolate distance filtering from tree aggregation, the most important missing evaluation. **Adaptive adversary:** the safety proof assumes static Byzantine assignment; defending against corruption that follows tree assignments requires VRF-based assignment with epoch rotation, described but not evaluated. **Cross-shard propagation:** the 95% propagation rate is consistent with typical gossip propagation at multi-second block intervals but not validated on a sharded testbed.

References

1. Bloom, B.H.: Space/time trade-offs in hash coding with allowable errors. Communications of the ACM **13**(7), 422–426 (1970)
2. Boneh, D., Lynn, B., Shacham, H.: Short signatures from the Weil pairing. Journal of Cryptology **17**(4), 297–319 (2004)

3. Broder, A.: On the resemblance and containment of documents. In: Proc. SEQUENCES (1997)
4. Buchman, E.: Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. Ph.D. thesis, University of Guelph (2016)
5. Castro, M., Liskov, B.: Practical Byzantine fault tolerance. In: Proc. OSDI (1999)
6. Charikar, M.: Similarity estimation techniques from rounding algorithms. In: Proc. STOC (2002)
7. Danezis, G., Kokoris-Kogias, L., Sonnino, A., Spiegelman, A.: Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. In: Proc. EuroSys (2022)
8. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.: Locality-sensitive hashing scheme based on p-stable distributions. In: Proc. SCG (2004)
9. Drake, J.: Pragmatic signature aggregation with BLS. <https://ethresear.ch/t/pragmatic-signature-aggregation-with-bls/2105> (2018)
10. Dwork, C., Lynch, N., Stockmeyer, L.: Consensus in the presence of partial synchrony. *Journal of the ACM* **35**(2), 288–323 (1988)
11. Ethereum Foundation: Danksharding. <https://ethereum.org/roadmap/danksharding>
12. Ethereum Foundation: Ethereum 2.0 beacon chain specification. <https://github.com/ethereum/consensus-specs>
13. Ethereum Foundation: py-ecc: Python BLS library. https://github.com/ethereum/py_ecc
14. Ethereum Foundation: Eth2 quick update no. 8. <https://blog.ethereum.org/2020/05/12/eth2-quick-update-no-8> (2020)
15. Golan-Gueta, G., Abraham, I., Grossman, S., Malkhi, D., Pinkas, B., Reiter, M.K., Seredinschi, D.A., Tamir, O., Tomescu, A.: SBFT: A scalable and decentralized trust infrastructure. In: Proc. DSN (2019)
16. Indyk, P., Motwani, R.: Approximate nearest neighbors: Towards removing the curse of dimensionality. In: Proc. STOC (1998)
17. Kotla, R., Alvisi, L., Dahlin, M., Clement, A., Wong, E.: Zyzzyva: Speculative Byzantine fault tolerance. In: Proc. SOSP (2007)
18. Mercier, R.: Distance-preserving digests: A primitive for BFT consensus (extended version). Technical report, <https://github.com/RyanMercier/Proxima> (2026)
19. Neiheiser, R., Matos, M., Rodrigues, L.: Kauri: Scalable BFT consensus with pipelined tree-based dissemination and aggregation. In: Proc. SOSP (2021)
20. Sigma Prime: Lighthouse update #27: BLS library performance. <https://lighthouse-blog.sigmaprime.io/update-27.html> (2020)
21. Skidanov, A.: Nightshade: Near protocol sharding design. <https://pages.near.org/papers/nightshade/>
22. Spiegelman, A., Giridharan, N., Sonnino, A., Kokoris-Kogias, L.: Bullshark: DAG BFT protocols made practical. In: Proc. CCS (2022)
23. Supranational: blst: BLS12-381 signature library. <https://github.com/supranational/blst>
24. Wagner, D.: A generalized birthday problem. In: Proc. CRYPTO. pp. 288–304 (2002)
25. Yin, M., Malkhi, D., Reiter, M.K., Golan-Gueta, G., Abraham, I.: HotStuff: BFT consensus with linearity and responsiveness. In: Proc. PODC (2019)